

Build a Weather AI Agent with Amazon Bedrock AgentCore — Modules 5–7

Step-by-step instructions for deploying the Agent Runtime, wiring up permissions, and deploying the frontend. Based on the official workshop, with corrections from a full run-through folded in.

Legend

■ **BUG / FIX** — a discrepancy found during testing, with the corrected step. These are **not** in the current lab guide (or contradict it).

■ **Checkpoint** — verify before moving on.

Module 5: Create Agent Runtime

The Agent Runtime hosts the AI agent code. You'll deploy the virtual meteorologist agent via **direct code deployment** — uploading a pre-built ZIP containing the Strands agent code and its ARM64 dependencies.

Complete the two tabs in order: create the IAM role first, then deploy the runtime.

Tab 1 — Create the Runtime IAM Role

The Agent Runtime needs an IAM role with permissions to invoke the Bedrock model, access the Gateway, use Memory, and read from S3. Create the policy first, then the role, then attach.

Step 1: Create the IAM Policy

1. In the **Services** search bar, search for and select **IAM**.
2. In the left panel, select **Policies**.
3. Click **Create policy**.
4. Select the **JSON** tab in the Policy editor.
5. Paste the following policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "BedrockModelAccess",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": [
        "arn:aws:bedrock:*::foundation-model/amazon.nova-2-lite-v1:0",
        "arn:aws:bedrock:*::inference-profile/us.amazon.nova-2-lite-v1:0"
      ]
    },
    {
      "Sid": "GatewayAccess",
      "Effect": "Allow",
      "Action": [
        "bedrock-agentcore:InvokeGateway",
        "bedrock-agentcore:GetGateway",
        "bedrock-agentcore:ListGatewayTargets"
      ],
      "Resource": "REPLACE_WITH_YOUR_GATEWAY_ARN"
    },
    {
      "Sid": "MemoryAccess",
```

```

    "Effect": "Allow",
    "Action": [
        "bedrock-agentcore:CreateEvent",
        "bedrock-agentcore:ListEvents",
        "bedrock-agentcore:GetMemory",
        "bedrock-agentcore>DeleteEvent"
    ],
    "Resource": "REPLACE_WITH_YOUR_MEMORY_ARN"
},
{
    "Sid": "S3CodeAccess",
    "Effect": "Allow",
    "Action": [
        "s3:GetObject",
        "s3:GetObjectVersion"
    ],
    "Resource": "arn:aws:s3:::bedrock-agentcore-runtime-*/*"
},
{
    "Sid": "CloudWatchLogs",
    "Effect": "Allow",
    "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogGroups",
        "logs:DescribeLogStreams"
    ],
    "Resource": "arn:aws:logs:us-east-1:*:log-group:/aws/bedrock-agentcore/runtimes/virtual_meteorologist*"
}
]
}

```

■ BUG / FIX — remove the Converse permissions.

The lab guide's `BedrockModelAccess` statement includes `bedrock:Converse` and `bedrock:ConverseStream`. These actions no longer exist — remove them from the `Action` list, keeping only `bedrock:InvokeModel` and `bedrock:InvokeModelWithResponseStream` (as shown in the corrected policy above). Leaving them in can cause the policy to be rejected.

Before saving, replace the placeholders with the actual ARNs you saved earlier:

Placeholder	Replace With	Where You Saved It
<code>REPLACE_WITH_YOUR_GATEWAY_ARN</code>	Your Gateway ARN (e.g. <code>arn:aws:bedrock-agentcore:us-east-1:123456789012:gateway/xxxxxxx</code>)	Module 3, Step 7
<code>REPLACE_WITH_YOUR_MEMORY_ARN</code>	Your Memory ARN (e.g. <code>arn:aws:bedrock-agentcore:us-east-1:123456789012:memory/virtual_meteorologist-...</code>)	Module 4, Step 5

6. Click **Next**.

7. For **Policy name**, enter a name (e.g. `agent-runtime-policy`).

8. Click **Create policy**.

Step 2: Create the IAM Role

1. In the left panel, select **Roles**.
2. Click **Create role**.
3. For **Trusted entity type**, select **Custom trust policy**.
4. Replace the default policy with:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "bedrock-agentcore.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

5. Click **Next**.

Step 3: Attach the Policy

1. Search for the policy you created in Step 1.
2. Select the checkbox next to it.
3. Click **Next**.

Step 4: Name and Create the Role

1. For **Role name**, enter a name (e.g. **agent-runtime-role**).
2. Click **Create role**.

■ **Checkpoint:** You've created the IAM role. Move to Tab 2 to deploy the runtime.

Tab 2 — Deploy the Runtime

Step 1: Download the Agent Runtime Package

Download the pre-built agent runtime ZIP (**agent-runtime.zip**). This contains the Strands agent code (**main.py**) and all required Python dependencies pre-packaged for ARM64.

Step 2: Navigate to AgentCore Runtime

1. In the **Amazon Bedrock AgentCore** console, select **Agent Runtime** under **Runtime** in the left panel.

■ **BUG / FIX — terminology changed.**

The button previously labeled **Host agent/tool** in the lab guide is now **Create runtime**. Click **Create runtime** instead.

Step 3: Configure the Agent

Setting	Value
Name	virtual_meteorologist

Under **Additional details**:

Setting	Value
Description	Virtual meteorologist agent powered by Amazon Nova 2 Lite with weather tools via AgentCore Gateway

Step 4: Configure the Agent Source

1. For the source type, select **Direct code deployment**.
2. For the package type, select **Upload ZIP**.
3. For the S3 path, leave the auto-populated value.
4. Click **Browse / Choose file** and select the `agent-runtime.zip` you downloaded.
5. For the entrypoint / handler, enter the value shown in the console.
6. For the architecture, select **ARM64**.

Note: The S3 bucket must be in the same region as your AgentCore Runtime (`us-east-1`). The console auto-creates this bucket for you.

Step 5: Configure Execution Role

1. For the execution role, select **Use an existing role**.
2. Select the role you created in Tab 1 (e.g. `agent-runtime-role`).

Step 6: Configure Environment Variables

Expand **Advanced configurations** at the bottom of the page. Under **Environment variables**, click **Add new variable** for each of the following:

Key	Value
<code>GATEWAY_ARN</code>	Gateway ARN saved in Module 3
<code>MEMORY_ID</code>	Memory ID saved in Module 4
<code>MODEL_ID</code>	<code>us.amazon.nova-2-lite-v1:0</code>
<code>AWS_REGION</code>	<code>us-east-1</code>

Leave **Security** set to **Public**.

Use the exact Gateway ARN and Memory ID you saved earlier — these are critical for the agent to connect to its tools and memory.

Step 7: Create the Agent

1. Review all settings.
2. Click **Create**.
3. Wait for the agent status to become **Active** (3–5 minutes).

Step 8: Save the Runtime ARN

- You may see: `"Update your IAM execution role with all resource IDs that are part of your Agent code."` You can safely dismiss this — the IAM policy from Tab 1 already includes the specific Gateway and Memory ARNs the agent needs.

- Once the runtime is active, copy the **Agent Runtime ARN** and save it to your notepad — you'll need it in Module 6.

Step 9 (Optional): Test the Agent

1. Select the endpoint and click **Test / Invoke**.
2. In the input section, replace the placeholder with your question:
`{"prompt": "What is the weather in Dallas, Texas?"}`
3. Click **Run** and observe the output.

If you see errors, check the CloudWatch logs under `/aws/bedrock-agentcore/runtimes/`.

■ **Checkpoint:** Agent Runtime deployed and active — direct code deploy via S3 ZIP, connected to the Gateway, Memory configured, Runtime ARN saved.

Module 6: Add Permissions to Identity Pool

Grant the Cognito Identity Pool's authenticated role permission to invoke the AgentCore Runtime. This is what lets the frontend call the agent on behalf of authenticated users.

Estimated time: ~5 minutes

Step 1: Navigate to the Identity Pool Role

1. In the **Services** search bar, search for and select **Amazon Cognito**.
2. In the left panel, select **Identity pools**.
3. Click on your identity pool.
4. Select the **User access** tab.

■ **BUG / FIX — assign the existing role.**

Under **User access**, set the authenticated role explicitly:

1. Click **Edit** on the authenticated role (or **User access** → **Edit role**).
2. Choose **Use an existing role**.
3. Select `cognito-identity-pool-iam-role`.
4. **Save changes**.
5. Scroll to the **Authenticated role** section and click the role name (`cognito-identity-pool-iam-role`) to open it in IAM.

Step 2: Add the AgentCore Permission

1. In the IAM console, select the **Permissions** tab.
2. Click **Add permissions** → **Create inline policy**.
3. Select the **JSON** tab in the Policy editor.
4. Paste the following policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeAgentCoreRuntime",
      "Effect": "Allow",
      "Action": "bedrock-agentcore:InvokeAgentRuntime",
```

```

    "Resource": [
      "REPLACE_WITH_YOUR_AGENT_RUNTIME_ARN",
      "REPLACE_WITH_YOUR_AGENT_RUNTIME_ARN/runtime-endpoint/DEFAULT"
    ]
  }
}
}

```

5. Replace **both** instances of `REPLACE_WITH_YOUR_AGENT_RUNTIME_ARN` with the Agent Runtime ARN you saved in Module 5. It should look like:

```
arn:aws:bedrock-agentcore:us-east-1:123456789012:runtime/virtual_meteorologist-XXXXXXX
XXX
```

You need both the runtime ARN **and** the runtime ARN with `/runtime-endpoint/DEFAULT` appended.

6. Click **Next**.

7. For **Policy name**, enter a name (e.g. `invoke-agent-runtime`).

8. Click **Create policy**.

■ **Checkpoint:** The Cognito Identity Pool's authenticated role can now invoke the AgentCore Runtime — inline policy added, scoped to your specific Agent Runtime ARN.

Module 7: Deploy Frontend & Test

Deploy the frontend with AWS Amplify and test the virtual meteorologist end-to-end.

Tab 1 — Deploy Amplify Application

Step 1: Download the Frontend Code

Download the pre-built frontend application (`AWS-Amplify-Frontend.zip`).

Step 2: Create the Amplify Application

1. In the **Services** search bar, search for and select **AWS Amplify**.
2. Click **Create new app / Deploy without Git**.
3. Select **Deploy without Git provider** and click **Next**.
4. Drag and drop the `AWS-Amplify-Frontend.zip` file.
5. Click **Save and deploy**.
6. Wait for deployment to complete (status changes from **Deploying** to **Deployed**).
7. Click the deployed **domain / URL** to open the application.

■ **BUG / FIX — Amplify trust policy needs updating (NOT in the lab guide).**

After Amplify is deployed, the Cognito Identity Pool role's **trust policy** references an outdated identity pool ID, which breaks authenticated sign-in. Fix it manually:

1. Go to the **IAM Console** → **Roles** and search for `cognito-identity-pool-iam-role` (path `service-role/`).
2. Open the role and click the **Trust relationships** tab.
3. Click **Edit trust policy**.
4. Find the line: `"cognito-identity.amazonaws.com:aud": "xxxxxxx"`

5. Replace the existing ID with the ID found in **Cognito** → **Identity pool** → **Identity pool ID**.
6. Save the trust policy.

Step 3: Configure the Application

The app opens with a configuration screen. Enter the values you saved throughout the workshop:

Amazon Cognito Configuration

Setting	Value
User Pool ID	User Pool ID from Module 1
User Pool Client ID	Client ID from Module 1
Identity Pool ID	Identity Pool ID from Module 1
Region	us-east-1

Amazon Bedrock AgentCore Configuration

Setting	Value
Agent Selection	Select AgentCore Agent from the dropdown
Agent Name	Virtual Meteorologist
AgentCore ARN	Agent Runtime ARN from Module 5
Region	us-east-1

Click **Save** to apply the configuration.

If the app can't connect, double-check every ID and ARN against your notepad.

Step 4: Log In

Use the test credentials created in Module 1 (username / password). On first login, you'll be prompted to change the password.

■ **Checkpoint:** The Amplify application is deployed and you're logged in.

Tab 2 — Test the Application

Try out the virtual meteorologist — test weather queries, conversation memory, and international locations.

Congratulations! Your agent can:

- ■ Understand natural language weather queries
- ■ Orchestrate multiple tools (geocoding, weather, date/time) automatically
- ■ Remember conversation context across messages
- ■ Provide friendly, emoji-rich weather responses

Summary of Bugs & Fixes

#	Module	Issue	Fix
---	--------	-------	-----

1	5 — Tab 1 (IAM Policy)	<code>bedrock:Converse</code> / <code>bedrock:ConverseStream</code> no longer exist	Remove both from the <code>BedrockModelAccess</code> policy statement
2	5 — Tab 2 (Deploy)	Button "Host agent/tool" renamed	Click Create runtime instead
3	6 — Identity Pool	Authenticated role not set	User access → Edit role → Use an existing role → <code>cognito-identity-pool-iam-role</code> → Save
4	7 — Amplify (not in guide)	Identity pool role trust policy has wrong <code>cognito-identity.amazonaws.com:aud</code> ID	IAM → Roles → <code>cognito-identity-pool-iam-role (service-role/)</code> → Trust relationships → Edit → replace <code>aud</code> ID with the Identity pool ID from Cognito